

Python Fundamentals

Part 4 — Object-Oriented Programming

Classes · Objects · Encapsulation · Inheritance · Abstraction · Polymorphism

1. What is OOP?

Object-Oriented Programming (OOP) is a way of organizing code around objects — bundles of data (attributes) and behavior (methods). Instead of writing loose functions and variables, we model real-world things as classes and create objects from them.

The four main pillars of OOP are:

1. **Encapsulation** — hiding internal data and controlling access
2. **Inheritance** — creating new classes from existing ones
3. **Abstraction** — hiding complex details, showing only essentials
4. **Polymorphism** — same method name, different behavior (overriding)

2. Classes & Objects

A class is a blueprint. An object is an instance created from that blueprint.

Syntax:

```
class ClassName:
    def __init__(self, ...):    # constructor
        self.attribute = value

    def method(self):
        # behavior
        pass

obj = ClassName(...)          # create object
```

self

self refers to the current object. It must be the first parameter of every instance method. Python passes it automatically when you call the method.

2.1 Example — BankAccount

```
class BankAccount:
    def __init__(self, account_number, owner_name, balance=0):
        self.account_number = account_number
        self.owner_name = owner_name
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount}. Balance: {self.balance}")
        else:
            print("Invalid amount")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew {amount}. Balance: {self.balance}")
        else:
            print("Insufficient funds or invalid amount")

    def check_balance(self):
```

```

        print(f"Balance: {self.balance}")
        return self.balance

# Usage
acc = BankAccount("A001", "Ali", 1000)
acc.deposit(500)
acc.withdraw(200)
acc.check_balance()    # 1300

```

2.2 Example — Book with Reviews

```

class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.reviews = []

    def add_review(self, review):
        self.reviews.append(review)
        print("Review added!")

    def count_reviews(self):
        return len(self.reviews)

    def display_reviews(self):
        if not self.reviews:
            print("No reviews yet.")
        else:
            for i, r in enumerate(self.reviews, 1):
                print(f"{i}. {r}")

book = Book("Python 101", "Shradha")
book.add_review("Great for beginners!")
book.add_review("Clear explanations.")
print(book.count_reviews())    # 2
book.display_reviews()

```

3. Encapsulation

Encapsulation means keeping data private and controlling how it is accessed or changed. In Python we use a leading underscore (`_`) to mark attributes as "private by convention".

We provide getter and setter methods so external code can read/write data only through controlled channels (with validation).

3.1 Example — Student with Validation

```

class Student:
    def __init__(self, name, roll_no, marks):
        self.set_name(name)
        self.set_roll_no(roll_no)
        self.set_marks(marks)

    # --- Getters ---
    def get_name(self):
        return self._name

    def get_roll_no(self):
        return self._roll_no

    def get_marks(self):
        return self._marks

```

```
# --- Setters with validation ---
def set_name(self, name):
    if name and name.strip():
        self._name = name.strip()
    else:
        raise ValueError("Name cannot be empty")

def set_roll_no(self, roll_no):
    if 1 <= roll_no <= 100:
        self._roll_no = roll_no
    else:
        raise ValueError("Roll number must be between 1 and 100")

def set_marks(self, marks):
    if marks >= 0:
        self._marks = marks
    else:
        raise ValueError("Marks cannot be negative")

s = Student("Ali", 25, 88)
print(s.get_name(), s.get_roll_no(), s.get_marks())
```

Naming Convention

`_name` (single underscore) = protected/private by convention. `__name` (double underscore) triggers name mangling. Prefer single underscore + getters/setters for clarity.

4. Inheritance

Inheritance lets a new class (child/subclass) reuse and extend an existing class (parent/base class). The child gets all attributes and methods of the parent.

```
class Parent:
    ...

class Child(Parent):    # Child inherits from Parent
    ...
```

4.1 Example — Vehicle Hierarchy

```
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display(self):
        print(f"{self.brand} {self.model}")

class Car(Vehicle):
    def __init__(self, brand, model, seats):
        super().__init__(brand, model)    # call parent constructor
        self.seats = seats

    def display(self):
        super().display()
        print(f"Seats: {self.seats}")

class Bike(Vehicle):
    def __init__(self, brand, model, engine_cc):
        super().__init__(brand, model)
        self.engine_cc = engine_cc

    def display(self):
        super().display()
```

```

        print(f"Engine: {self.engine_cc} cc")

car = Car("Toyota", "Corolla", 5)
car.display()
bike = Bike("Honda", "CBR", 250)
bike.display()

```

super()

super() lets you call methods of the parent class. Use it in `__init__` to initialize parent attributes, and when overriding methods to reuse parent logic.

5. Method Overriding (Polymorphism)

When a subclass defines a method with the same name as one in the parent, it overrides the parent's version. Different objects can respond differently to the same method call — this is polymorphism.

5.1 Example — Shape Hierarchy

```

import math

class Shape:
    def area(self):
        raise NotImplementedError("Subclass must implement area()")

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height

shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]
for s in shapes:
    print(f"{type(s).__name__} area: {s.area():.2f}")

```

6. Abstraction

Abstraction hides complex implementation details and exposes only what is necessary. In Python we use the `abc` module to create abstract base classes (ABC) with abstract methods that subclasses must implement.

6.1 Example — Employee Salary

```

from abc import ABC, abstractmethod

class Employee(ABC):

```

```

def __init__(self, name):
    self.name = name

    @abstractmethod
    def calculate_salary(self):
        pass

class Intern(Employee):
    def __init__(self, name, stipend):
        super().__init__(name)
        self.stipend = stipend

    def calculate_salary(self):
        return self.stipend

class FullTimeEmployee(Employee):
    def __init__(self, name, basic, bonus):
        super().__init__(name)
        self.basic = basic
        self.bonus = bonus

    def calculate_salary(self):
        return self.basic + self.bonus

class ContractEmployee(Employee):
    def __init__(self, name, hourly_rate, hours):
        super().__init__(name)
        self.hourly_rate = hourly_rate
        self.hours = hours

    def calculate_salary(self):
        return self.hourly_rate * self.hours

intern = Intern("Sara", 15000)
full = FullTimeEmployee("Ali", 50000, 10000)
contract = ContractEmployee("Omar", 500, 80)

for e in [intern, full, contract]:
    print(f"{e.name}: {e.calculate_salary()}")

```

Important

You cannot create an instance of an abstract class. Subclasses must implement every abstract method, or they remain abstract themselves.

7. Constructor Overloading (Default Parameters)

Python does not allow multiple `__init__` methods in the same class. We simulate constructor overloading using default parameter values.

7.1 Example — Person

```

class Person:
    def __init__(self, name, age=None, address=None):
        self.name = name
        self.age = age
        self.address = address

    def display(self):
        print(f"Name: {self.name}")
        if self.age is not None:
            print(f"Age: {self.age}")
        if self.address is not None:
            print(f"Address: {self.address}")

```

```

p1 = Person("Ali")           # name only
p2 = Person("Sara", 22)      # name + age
p3 = Person("Omar", 30, "Dhaka") # name + age + address
p1.display()
p2.display()
p3.display()

```

8. Instance & Class Attributes

Instance attributes belong to each object (e.g. name, level). Class attributes are shared by all objects of the class (e.g. a counter).

Type	Defined where	Belongs to	Example
Instance attr	Inside <code>__init__</code> (self.x)	One object	<code>self.name</code>
Class attr	Directly in class body	All objects	<code>player_count = 0</code>

8.1 Example — Player Counter

```

class Player:
    player_count = 0          # class attribute

    def __init__(self, name, level=1):
        self.name = name      # instance attributes
        self.level = level
        Player.player_count += 1

    def display(self):
        print(f"{self.name} (Level {self.level})")

    @classmethod
    def get_count(cls):
        return cls.player_count

p1 = Player("Apex")
p2 = Player("Nova", 5)
p3 = Player("Blitz", 3)
print(Player.get_count())    # 3
print(Player.player_count)   # 3

```

9. Multiple Inheritance

A class can inherit from more than one parent. Python supports this directly.

```

class Parent1:
    ...
class Parent2:
    ...
class Child(Parent1, Parent2):
    ...

```

9.1 Example — Bear (Herbivore + Carnivore + Omnivore)

```

class Herbivore:
    def eat_plants(self):
        print("Eating plants / berries")

class Carnivore:
    def eat_meat(self):
        print("Eating meat / fish")

```

```

class Omnivore:
    def eat_anything(self):
        print("Can eat both plants and meat")

class Bear(Herbivore, Carnivore, Omnivore):
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(f"I am {self.name}, a bear.")

b = Bear("Baloo")
b.introduce()
b.eat_plants()      # from Herbivore
b.eat_meat()        # from Carnivore
b.eat_anything()    # from Omnivore

```

MRO (Method Resolution Order)

When the same method exists in multiple parents, Python follows a defined order (MRO). Check it with `ClassName.__mro__` or `help(ClassName)`.

10. Mini Project — OOP Chat System

Putting it all together: classes `User`, `Message`, and `ChatRoom` that support joining, leaving, sending messages, and viewing history.

```

from datetime import datetime

class User:
    def __init__(self, username):
        self.username = username

    def __str__(self):
        return self.username

class Message:
    def __init__(self, sender, content):
        self.sender = sender
        self.content = content
        self.timestamp = datetime.now().strftime("%H:%M:%S")

    def __str__(self):
        return f"[{self.timestamp}] {self.sender}: {self.content}"

class ChatRoom:
    def __init__(self, name):
        self.name = name
        self.users = []
        self.messages = []

    def join(self, user):
        if user not in self.users:
            self.users.append(user)
            print(f"{user} joined {self.name}")

    def leave(self, user):
        if user in self.users:
            self.users.remove(user)
            print(f"{user} left {self.name}")

    def send(self, user, content):
        if user not in self.users:
            print(f"{user} is not in the room")
        return

```

```

        msg = Message(user, content)
        self.messages.append(msg)
        print(msg)

    def history(self):
        print(f"--- Chat history of {self.name} ---")
        for m in self.messages:
            print(m)

# Demo
room = ChatRoom("Python Lounge")
u1 = User("Ali")
u2 = User("Sara")
room.join(u1)
room.join(u2)
room.send(u1, "Hello everyone!")
room.send(u2, "Hi Ali!")
room.history()
room.leave(u1)

```

11. Quick Summary

Concept	Key Idea	Python Tool
Class / Object	Blueprint → instance	class, obj = Class()
Encapsulation	Hide data, control access	_attr, getters, setters
Inheritance	Child reuses parent	class Child(Parent)
Overriding	Same method, new behavior	def method(self) in child
Abstraction	Force subclasses to implement	ABC, @abstractmethod
Constructor overload	Flexible __init__	Default parameters
Class attribute	Shared by all objects	Defined in class body
Multiple inheritance	Inherit from many parents	class C(A, B)

Keep Learning & Keep Exploring!